Booleovská algebra. Podmíněný příkaz.

Booleovská funkce. Booleovské zákony. Příkaz if/else.

Tomáš Bayer | bayertom@natur.cuni.cz

Katedra aplikované geoinformatiky a kartografie, Přírodovědecká fakulta UK.

Obsah přednášky

1. Booleovská funkce
2. Přehled unárních booleovských funkcí
3. Přehled binárních booleovských funkcí
4. Základní zákonitosti při práci s Booleovskými funkcemi
5. Stavební prvky algoritmu
6. Blok příkazů
   - Příkaz if-else
   - Ternární operátor
   - Příkaz match-case

1. Úvod

Spojité veličiny:
Proměnné veličiny, které nabývají “nekonečného” množství hodnot.
Lze je popsat spojitými proměnnými a vyjádřit reálnými datovými typy.

Diskrétní veličiny:
Proměnné veličiny, které nabývají konečného množství hodnot.
Lze je popsat diskrétními proměnnými a vyjádřit celočíselnými datovými typy.

Booleovská algebra:
Proměnné veličiny nabývají hodnot 0,1.
Hodnotu 0 lze interpretovat jako nepravda, hodnotu 1 jako pravda.
V informatice hraje velmi výraznou roli při konstrukci relací.

2. Booleovská funkce

Booleovská funkce n proměnných $f : B^n \to B$,

$$y = f(x_1, ..., x_n), \qquad x_i, y \in B.$$

Unární funkce, $n = 1$, $f : B^1 \to B$,

$$y = f(x_1).$$

Jednoprvková booleovská algebra

$$B^1 = \{0, 1\}^1.$$

Binární funkce, $n = 2$, $f : B^2 \to B$,

$$y = f(x_1, x_2).$$

Dvouprvková booleovská algebra

$$B^2 = \{0, 1\}^2 = \{0, 1\} \times \{0, 1\} = \{\{0, 0\}, \{0, 1\}, \{1, 0\}, \{1, 1\}\}.$$

Pravdivostní tabulka:
Schematické znázornění prvků a funkčních hodnot.
Sousední prvky se liší právě o 1 hodnotu.

x_1	y
x_1'	$f(x_1')$
x_1	$f(x_1)$

x_1	x_2	y
x_1'	x_2'	$f(x_1', x_2')$
x_1'	x_2	$f(x_1', x_2)$
x_1	x_2'	$f(x_1, x_2')$
x_1	x_2	$f(x_1, x_2)$

3. Unární booleovská funkce

Unární booleovská funkce $f : B^1 \to B$

$$y = f(x_1), \qquad x_1, y \in B.$$

Definiční obor unární funkce dán 2^1 hodnotami.
Existuje 2^{2^1} unárních booleovských funkcí.

x_1	$f_1(x_1)$	$f_2(x_1)$	$f_3(x_1)$	$f_4(x_1)$
0	0	1	0	1
1	0	0	1	1

Přehled unárních booleovských funkcí:

- Falsum.
- Negace.
- Aserce.
- Verum.

V programování používána negace.

4. Přehled unárních booleovských funkcí

Falsum

$$f_1(x_1) \to \{0\}.$$

Libovolné hodnotě x přiřazuje hodnotu False.

Negace

$$f_2(x_1) \quad = \quad \begin{cases} 1, & \text{pro } x_1 = 0, \\ 0, & \text{pro } x_1 = 1. \end{cases}$$

Nejznámější a nejčastěji používaná unární funkce.
Přiřazuje $f(x_1)$ opačnou hodnotu než x_1 ($\overline{x}_1$ doplňek k x_1).
Označována:′ ¯, !, not, ~.

Aserce

$$f_3(x_1) \quad = \quad \begin{cases} 1, & \text{pro } (x_1) = 1, \\ 0, & \text{pro } (x_1) = 0. \end{cases}$$

Aserce přiřazuje hodnotě $f(x_1)$ hodnotu proměnné x_1.

Verum $y = f_4(x)$

$$f_4(x_1) \to \{1\}.$$

Libovolné hodnotě x_1 přiřazuje hodnotu True.

5. Binární booleovská funkce

Booleovská funkce $f : B^2 \to B$ dvou proměnných

$$y = f(x_1, x_2), \qquad x_1, x_2, y \in B.$$

Definiční obor binární funkce dán 2^2 hodnotami.
Existuje $2^{2^2} = 16$ binárních booleovských funkcí.

Nejčastěji používané booleovské binární funkce:

- Konjunkce.
- Disjunkce.
- Negace konjunkce.
- Negace disjunkce.
- Ekvivalence.
- Nonekvivalence.
- Implikace.

V informatice používány nejčastěji konjunkce a disjunkce.

6. Přehled Booleovských funkcí v B^2

x_1	x_2	f_1	f_2	f_3	f_4	f_5	f_6	f_7	f_8
0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

x_1	x_2	f_9	f_{10}	f_{11}	f_{12}	f_{13}	f_{14}	f_{15}	f_{16}
0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1

7. Konjunkce

Tzv. logický součin.
Popsána pravdivostní tabulkou

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	0
1	0	0
1	1	1

Funkční hodnota $y = 1$, pokud proměnné $x_1 = x_2 = 1$.
Označení $\wedge$, $\cdot$, logický operátor AND.
Zápis $a \wedge b$ čteme jako "`a i b`".

V programovacích jazycích: and, &&, &.

```
x2 = (0, 1, 0, 1)
x1 = (0, 0, 1, 1)
y = (x1[0] & x2[0], x1[1] & x2[1], x1[2] & x2[2], x1[3] & x2[3]
```

8. Disjunkce

Tzv. logický součet.
Popsána pravdivostní tabulkou

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	1

Poslední řádkem tabulky se disjunkce liší od “obyčejného” součtu.

Funkční hodnota $y = 0$, pokud $x_1 = x_2 = 0$..
Označení $\vee$, $+$, logický operátor OR.
Zápis $a \vee b$ čteme jako “`a nebo b`”.

V programovacích jazycích: or, ||, |.

```
x2 = (0, 1, 0, 1)
x1 = (0, 0, 1, 1)
y = (x1[0] | x2[0], x1[1] | x2[1], x1[2] | x2[2], x1[3] | x2[3]
```

9. Negace konjunkce

Tzv. Schefferova funkce.
Popsána pravdivostní tabulkou

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	1
1	0	1
1	1	0

Funkční hodnota $y = 1$, pokud $x_1 \neq 1 \vee x_2 \neq 1$.

Vyjádření operátorem NAND.

10. Negace disjunkce

Tzv. Pierceova funkce.
Popsána pravdivostní tabulkou

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	0
1	0	0
1	1	0

Funkční hodnota $y = 1$ pokud $x_1 \neq 0 \wedge x_2 \neq 0$.

Vyjádření operátorem NOR.

11. Ekvivalence

Popsána pravdivostní tabulkou

x_1	x_2	$f(x_1, x_2)$
0	0	1
0	1	0
1	0	0
1	1	1

Funkční hodnota $y = 1$ pokud $x_1 = x_2$.
Porovnání dvou výroků, zda mají stejnou pravdivostní hodnotu.

Vyjádření operátorem Iff.

12. Nonekvivalence

Popsána pravdivostní tabulkou

x_1	x_2	$f(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

Analogie sčítání, označení $\oplus$.
Funkční hodnota $y = 1$ pokud $x_1 \neq x_2$.

Vyjádření operátorem XOR" (eXlucive OR).

13. Priorita booleovských operací

Vyhodnocování výrazu z leva do prava.
Různá priorita (pořadí vyhodnocení) booleovských funkcí.
Analogie s běžnými aritmetickými operacemi.

Priorita booleovských operací:

1. Negace.
2. Konjunkce.
3. Disjunkce.
4. Ostatní.

Změna priorit operací:
Prostřednictvím závorek, obdoba aritmetických výrazů.

Příklad: platí ekvivalence?:
$f(x_1, x_2, x_3, x_4) : x_1 + x_2 \cdot x_3 + x_4 \neq (x_1 + x_2) \cdot (x_3 + x_4)$.
$f(x_1, x_2, x_3, x_4) : x_1 \cdot x_2 + x_3 \cdot x_4 = (x_1 \cdot x_2) + (x_3 \cdot x_4)$.

14. Booleovské unární zákony

Zákon dvojí negace (Double Negation Law):

$$\left(x'\right)' = x. \tag{1}$$

Zákon vyloučení třetí možnosti (Complement Law):

$$x' + x = 1. \tag{2}$$

Zákon sporu (Complement Law):

$$x' \cdot x = 0. \tag{3}$$

Zákony opakování (Idempotent Law):

$$x + x = x, \tag{4}$$

$$x \cdot x = x. \tag{5}$$

Zákony neutrálnosti (Identity Law):

$$x + 0 = x, \tag{6}$$

$$x \cdot 1 = x. \tag{7}$$

Zákony agresivnosti (Annulment Law):

$$x + 1 = 1, \tag{8}$$

$$x \cdot 0 = 0. \tag{9}$$

15. Booleovské binární zákony (1/2)

Komutativní zákony (Commutative Laws):

$$x_1 + x_2 = x_2 + x_1, \tag{10}$$
$$x_1 \cdot x_2 = x_2 \cdot x_1. \tag{11}$$

Asociativní zákony (Associative Laws):

$$x_1 + (x_2 + x_3) = (x_1 + x_2) + x_3 = x_1 + x_2 + x_3, \tag{12}$$
$$x_1 \cdot (x_2 \cdot x_3) = (x_1 \cdot x_2) \cdot x_3 = x_1 \cdot x_2 \cdot x_3. \tag{13}$$

Distributivní zákony (Distributive Laws):

$$x_1 \cdot (x_2 + x_3) = x_1 \cdot x_2 + x_1 \cdot x_3, \tag{14}$$
$$x_1 + (x_2 \cdot x_3) = (x_1 + x_2) \cdot (x_1 + x_3). \tag{15}$$

16. Booleovské binární zákony (2/2)

De Morganovy zákony (de Morgan´s Theorem):

$$(x_1 + x_2)' = x_1' \cdot x_2', \tag{16}$$
$$(x_1 \cdot x_2)' = x_1' + x_2'. \tag{17}$$

Absorpční zákony (Absorptive Law):

$$x_1 + (x_1 \cdot x_2) = x_1, \tag{18}$$
$$x_1 \cdot (x_1 + x_2) = x_1. \tag{19}$$

Zákony absorpce negace (Absorptive Law with Negation):

$$x_1 + (x_1' \cdot x_2) = x_1 + x_2, \tag{20}$$
$$x_1 \cdot (x_1' + x_2) = x_1 \cdot x_2. \tag{21}$$

Důkaz, Absorptive Law*:*

$$\begin{aligned} x_1 + (x_1 \cdot x_2) &= (x_1 \cdot 1) + (x_1 \cdot x_2), \| \ 9 \\ &= x_1 \cdot (1 + x_2), \| \ 14 \\ &= x_1 \cdot 1, \| \ 8 \\ &= x_1. \end{aligned}$$

17. Minimalizace booleovské funkce

Vycházíme z boolevských zákonů.
Každou booleovskou funkci lze vyjádřit prostřednictvím konjunkce, disjunkce, negace (lin. kombinace).

Příklad 1:

$$\begin{aligned} y = f(x_1, x_2, x_3) &= x_1' \cdot x_2' \cdot x_3' + x_1 \cdot x_2' \cdot x_3', \\ &= x_2' \cdot x_3'(x_1 + x_1'), \parallel 2 \\ &= x_2' \cdot x_3' \parallel 7. \end{aligned}$$

Příklad 2:

$$\begin{aligned} y = f(x_1, x_2) &= [(x_1 \cdot x_2)' + (x_1 \cdot x_2)]' + x_1, \\ &= [(x_1 \cdot x_2)']' \cdot (x_1 \cdot x_2)' + x_1 \parallel (18), \\ &= (x_1 \cdot x_2) \cdot (x_1 \cdot x_2)' + x_1 \parallel (2), \\ &= 0 + x_1 \parallel (4), \\ &= x_1. \end{aligned}$$

Příklad 3:
Mohu volit, pokud mi bylo >18 let, a mohu prokázat totožnost. Negujte.
Chci si koupit Iphone nebo Ipad. Negujte.

18. Tautologie

Booleovská formule, je vždy pravdivá

$$f(x_1, ..., x_n) \rightarrow \{1\}.$$

Pro libovolnou kombinaci argumentů nabývá hodnoty pravda.
Unární / binární zákony představují tautologie.

Používány při posuzování identity vztahů, důkazech, atd.
Jednoduchá tautologie (Complement Law)

$$x' + x = 1$$

Podmínky v cyklu/rekurzi: nutno se vyhnout tautologiím.
Jinak nekonečný cyklus/rekurze.

19. Kontradikce

Booleovská formule, je vždy nepravdivá

$$f(x_1, ..., x_n) \to \{0\}.$$

Pro libovolnou kombinaci argumentů nabývá hodnoty nepravda.
Kontradikce je negací tautologie.
Tautologie je negací kontradikce.

Jednoduchá kontradikce (Annulment Law)

$$x' \cdot x = 0.$$

Při navrhování podmínek v cyklu/rekurze se vyhnout kontradikcím.
Podmínka/cyklus/rekurze se neprovedou.

20. Příklad

Ověřte, zda $f(x)$ je tautologie/kontradikce:

$$\begin{aligned} f(x) &= x \cdot (x + (x + x'))', \\ &= x \cdot (x + 1)' \parallel 3, \\ &= x \cdot (1') \parallel 9, \\ &= x \cdot 0 \parallel 10, \\ &= 0. \end{aligned}$$

Funkce $f(x)$ je kontradikcí.
Ověřte, zda $f(x_1, x_2)$ je tautologie/kontradikce:

$$\begin{aligned} f(x_1, x_2) &= \left[((x_1 \cdot x_2) + x_1)' \cdot (x_1 \cdot (x_2 + x_1))\right]', \\ &= \left[((x_1 \cdot x_2) + x_1)' \cdot (x_1)\right]' \parallel 20, \\ &= (x_1' \cdot x_1)' \parallel 19, \\ &= 0' \parallel 4, \\ &= 1. \end{aligned}$$

Funkce $f(x_1, x_2)$ je tautologií.

21. Stavební prvky programu

ákladní stavební prvky programu, příkazy:

- *Jednoduché příkazy*
  Základní stavební jednotka programu, elementární.
  Prázdný příkaz: `pass`
  Přiřazovací příkaz: `=`
  Příkaz skoku: `break, continue`
  Příkaz podprogramu (procedury, funkce): `def`
- *Strukturované příkazy*
  Tvořeny jednoduchými či dalšími příkazy.
  Složený příkaz (blok): odsazení
  Příkaz pro větvení programu (podmínka): `if-else, match-case`
  Příkaz pro opakování (cyklus): `for, while`

Vzájemně mohou být kombinovány.

Implementovány prakticky ve všech programovacích jazycích.
Syntaxe +- podobná.

22. Blok příkazů

Skupina kroků, které k sobě logicky patří, a vykonají se jako celek.
Odsazení: příslušnost k bloku.

```
Příprava čaje:
    Ohřej vodu v konvici.
    Dej sáček čaje do hrnku.
    Zalij sáček horkou vodou.
```

Jednotlivé kroky mohu/nemusí být elementární.
Obsahuje libovolný počet příkazů.

Použití s klíčovými slovy: `if`, `def`, `for`, `while`

```
if D > 0:                  # Nasleduje blok
    x1 = (-b + sqrt(D)) / (2*a)
    x2 = (-b - sqrt(D)) / (2*a)
```

C/C++/Java: použity {}.

```
if (D > 0){                  # Nasleduje blok
    x1 = (-b + sqrt(D)) / (2*a)
    x2 = (-b - sqrt(D)) / (2*a)
}
```

23. Příkazy pro větvení programu

Často označovány jako řídící struktury.
Patří k nejčastěji používaným konstrukcím.
Reagují na situace, ke kterým dochází v průběhu běhu programu.

Bývají nazývány podmíněnými příkazy: něco se koná - když.
Realizují větvení algoritmu do více větví.
O tom, která větev se vykoná, rozhoduje hodnota booleovského výrazu.

Typy podmínek:

- neúplná,
- úplná,
- kombinovaná.

Syntakticky podobné ve většině programovacích jazyků: `if-else`.

Podpora `match-case` (Python 3.10+).

24. Neúplná podmínka

Booleovský výraz tvoří řídící podmínku.
Pokud je vyhodnocena jako pravdivá, provede se příkaz/blok příkazů.

```
if booleovsky_vyraz: #Pokud splneno, proved prikazy v bloku
   prikaz1
   prikaz2
   ...
```

Neřeší se, co dělat v případě nesplnění podmínky.
V praxi neúplná podmínka používána spíše u cyklu.
Podmínku uvádět v “kladném” tvaru, ne v negaci!

Příklad:

```
if t > 35:
   print ('Swim')
```

Vnořená podmínka: podmínka uvnitř těla jiné podmínky:

```
if t > 35:
   if holiday:              #Vnorena podminka
     print ('Swim')

if (t > 35) and (holiday):  #Spojeni obou podminek do jedne
   print ('Swim')
```

25. Úplná podmínka

Řešeny obě varianty: kladná i záporná.
Vznikne rozšířením neúplné podmínky o konstrukci `else` + blok.

Říkáme, co se bude dít při splnění i nesplnění podmínky.
Ve větvi else neprovádíme před vykonáním bloku testování.

```
if booleovsky_vyraz: #Pokud splneno, proved prikazy v tomto bloku
   prikaz1
   ...
else:                #Jinak proved prikazy v tomto bloku
   prikaz2
   ...
```

Příklad: Ukázka úplné podmínky

```
if t > 35:
   print ('Swim')
else:
   print ('Paddle')
```

Varianta buď a nebo.
Else vždy párováno s nejbližším if.

26. Kombinovaná podmínka

Dvě varianty mnohdy nestačí, výběr z více variant.
Varianty se vzájemně vylučují.
Každá, s výjimkou poslední, testována.

Úplná podmínka doplněná o další varianty.

Umožňuje realizovat složitější větvení programu: více než 2 varianty.
Použit příkaz `elif`, zkrácení `else if`.

```
if booleovsky_vyraz1:      #Otestuj 1. podminku
   prikaz1
   ...
elif booleovsky_vyraz2:    #Pokud nesplnena, otestuj 2. podminku
   prikaz2
   ...
elif booleovsky_vyraz3:    #Pokud nesplnena, otestuj 3. podminku
   prikaz3
   ...
else:                      #Default, pokud nesplnena zadna z predchozic
   prikaz4
   ...
```

Pozor na pořadí podmínek: rozšiřující, ne zužující.

27. Kombinovaná podmínka, příklad

Rozšiřující podmínka:
Množina hodnot, ze které vybíráme, se zvětšuje (OK).

```
if t > 35:
   print ('Swim')
elif t > 30:
   print ('Paddle')
elif t>25:
   print ('Walk')
else:
   print ('Rest')
```

Zužující podmínka:
Množina hodnot, ze které vybíráme, se zmenšuje (Chyba).

```
if t>25:
   print ('Walk')
elif t > 30:
   print ('Paddle')
elif t > 35:
   print ('Swim')
else:
   print ('Rest')
```

28. Ternární operátor

Umožňuje zapsat úplnou podmínku stručnějším způsobem.
Operátor má tři argumenty: 2 hodnoty a výraz.

```
hodnota1 if condition else hodnota2;
```

Je -li výraz vyhodnocen jako pravdivý, je vrácena `hodnota1`, jinak `hodnota2`.

Nevýhoda: nižší přehlednost.

Podmínku

```
if a < b:
   c = a + 10
else:
   c = a - 10;
```

lze zapsat jako

```
c = a+10 if a<b else a-10
c = (a+10) if a<b else (a-10)
```

Závorky nepovinné, slouží pro zdůraznění podmínky.

29. Příkaz match-case

Přepínač, umožňuje větvení programu do více větví.
Počet větví není omezen, v každé vykonán příkaz/blok.
Přehlednější varianta `if-elif-else`.

```
match vyraz:                        #Vyraz
        case const1:                #Navesti 1
            prikaz
        case const2:                #Navesti 2
            prikaz
        case _:                     #Navesti 3, vychozi, nepovinne
            prikaz
```

Vyhodnocením výrazu celočíselná hodnota.
Návěští: celočíselná /znaková, unikátní hodnota.
Dle hodnoty výrazu nalezeno odpovídající návěští + provedeny příkazy v jeho bloku.

```
match x:
        case 'a':               #if
            return 0
        case 'b':               #elif
            return 1
        case _:                 #else
            return 2
```

Pokud nenalezeno odpovídající návěští, vykonán kód nacházející v návěští `default`.

30. Příklad: kvadratická rovnice

```
import math
from math import sqrt

a = int(input("a: "))   #Standardni vstup
b = int(input("b: "))
c = int(input("c: "))

D = b * b - 4. * a * c  #Diskriminant

if D < 0:    #0 reseni
    print ("Nema reseni v R.")

elif D == 0: #1 reseni
    x = (-b + sqrt(D))/(2. * a)
    print ("Dvojnasobny koren: ", x)

else:        #2 reseni
    x1 = (-b - sqrt(D))/(2. * a)
    x2 = (-b + sqrt(D))/(2. * a)
    print ("Dva koreny: ", x1, " a", x2)
```